

我们在为一家互联网电商开发订单处理软件，该公司从供应商那里购买产品，然后销售给客户。这家公司在线发布商品目录，并将其推送给客户和其他感兴趣的人。

客户以提交商品列表并向电商付费的方式购买商品。电商填写帐单，并委托快递公司把商品运送到客户的地址。订单处理软件记录从收到订单直到商品被运送给客户的整个过程。电商将提供快捷的服务，以最快、最有效的方法来发送客户订购的产品。客户可以退货，但有时要付运费。

电商订单处理软件系统的参与者不包括：

提交

- A 网络电商
- B 客户
- C 其它感兴趣的人
- D 快递公司



在网络电商客户定单处理应用中与客户有关的用例不包括：

- A 退货
- B 计算运费
- C 浏览商品
- D 订单查询

提交

进入订购商品用例的前置条件是：

提交

- A 客户对商品感兴趣
- B 客户安装了与系统兼容的浏览器版本
- C 商品已经放入购物车
- D 客户通过合法账户登入系统

使用订单处理系统一段时间以后，电商希望增加一种功能——为老顾客提供折扣。以下哪种方法比较合适？

提交

- A 建立老顾客折扣新用例
- B 扩展订购商品用例
- C 在订购商品用例中包含老顾客提供折扣用例
- D 为订购商品用例建立两个子用例：普通顾客订购商品和老顾客订购商品



面向对象分析 Object-Oriented Analysis

李起成

College of Computer Science
Nankai University

*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University) and
Ivan Marsic (Rugers University)*



南开大学
Nankai University





Object-Oriented Analysis

- 面向对象分析技术关注应用领域中的实体，并将其建模为对象
- 面向对象分析技术主要基于分类、泛化、聚合关系，在对象集合之间建立结构
- 对象的行为是执行预定的动作（服务/活动）
- 对象通过执行动作来完成状态变迁



面向对象分析方法举例

- Peter Coad的面向对象方法 [Coad91]
- “对象” 是问题领域中真实存在的实体， 有“定义清晰的边界”
- 对象中封装有属性和行为
- 面向对象分析的五个核心概念：
 - 对象、属性、结构、服务和主题



类 标 识 符

患 者

属性

患者姓名
家庭住址
医师姓名

服务

预约



服务建模 (Services)

- 对象为其周遭的其他对象提供服务，例如，医生对象对外提供的服务包括：体检，出体检报告等。
- Coad 的OOA方法中，定义了三种类型的服务：
 - **瞬时服务** (Occurrence services)：对象的创建、结束，修改等等
 - **计算服务** (Calculate services)：对象为其他对象完成计算任务等
 - **监控服务** (Monitor services)：对象持续监控流程，检查预设条件是否满足
- 我们用带箭头的虚线来表示一个对象引用另一个对象的服务

从面向对象到面向服务，是看待问题的视角的切换

软件系统建模原则

- 支持修改和重用
 - **有经验的工程师重用已有的设计**
 - 模型组件重用
 - 模型结构重用
 - **有智慧的工程师规划未来**
 - 建立可重用的模型组件
 - 建立易于修改的模型结构
- 五大有力武器:
 - **抽象**: 关注重点, 暂忘细节
 - **分解**: 将问题切分, 分而治之
 - **多视角映射**: 切换关注点, 分别讨论
 - **模块化**:
 - 建立稳定结构、缩小变化影响范围
 - **模式**: 多次成功应用的模型结构

对象建模原则1: 抽象

- 抽象
- **在对象间找出共性，忽略不相关细节**
 - **关注对象间的一般/特殊关系**
 - 将具有相同属性或角色的对象放入同一个类集合中
 - 再通过父子关系，将由共性的类定义为同一个父类的子类
- 例如:
 - 需求是处理航天器故障
 - 将故障按不同的故障类型纵向分类。

based on location:

- ↳ instrumentation fault,
- ↳ communication fault,
- ↳ processor fault,
- ↳ etc

OR

based on symptoms:

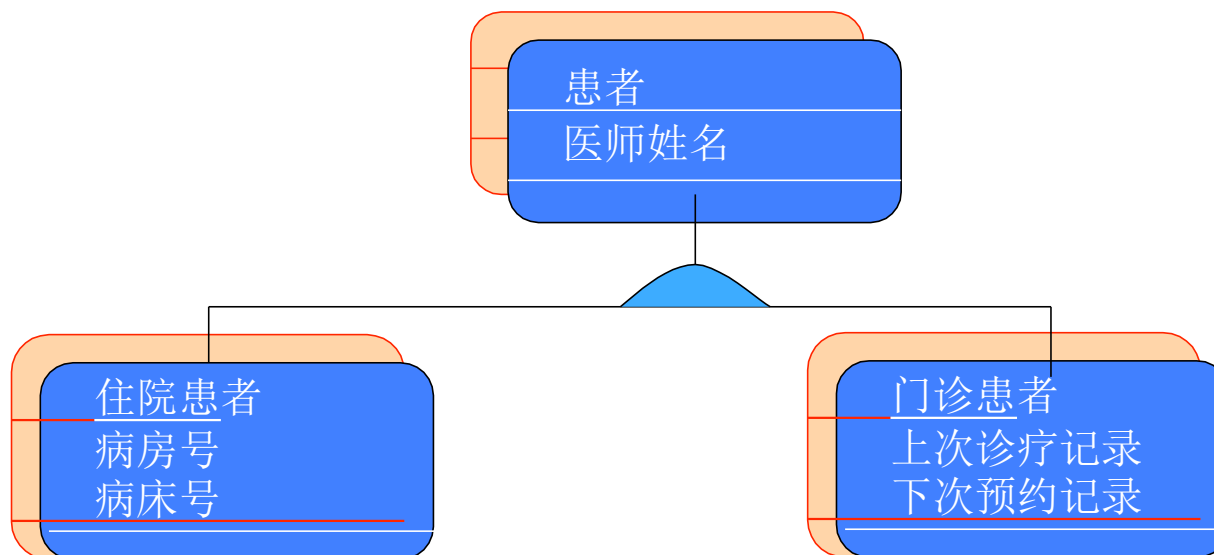
- ↳ no response from device;
- ↳ incorrect response;
- ↳ self-test failure;
- ↳ etc...



继承/一般-特殊结构 (Gen-Spec Structures)

一般-特殊结构将类组织成基于继承关系的分类层次结构

- 自底向上是从特殊到一般的类 (generalization)
- 自顶向下是从一般到特殊的类 (specialization)



对象建模原则2：分解

- 分解
 - 表达整体部分关系
- 例如：
 - 目标是飞行器研发
 - 将问题分解为子系统研发：
 - 导航系统;
 - 数据处理系统;
 - 指挥控制系统;
 - 环境控制系统;
 - 等

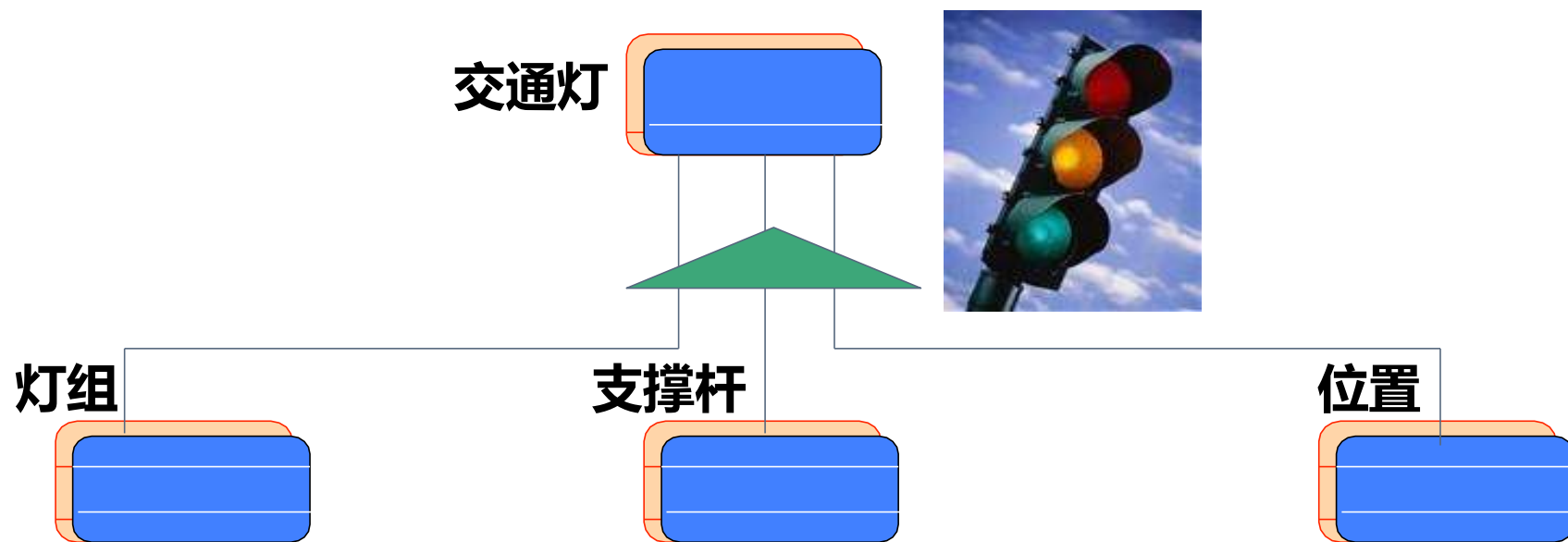
这是我们对问题的分解

- 现实世界中的设计可以组件化
- 系统分解方式决定系统的体系结构设计



整体-部分结构 (Whole-Part Structures)

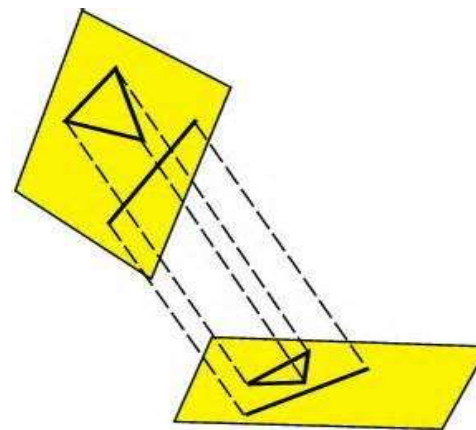
- 整体-部分结构描述对象间的组合关系. 例如, 一个交通灯对象由 0-3 个灯组, 以及支撑杆和位置对象组合而成。



对象建模原则3：投影

- 投影:
 - **从多个视角分别建模问题的不同方面**
 - 一如建筑施工中的不同视角的图纸

- 例如:
 - **需要进行飞行器需求建模**
 - **投影建议分别建模:**
 - 安全性、容错性、实时性 ...



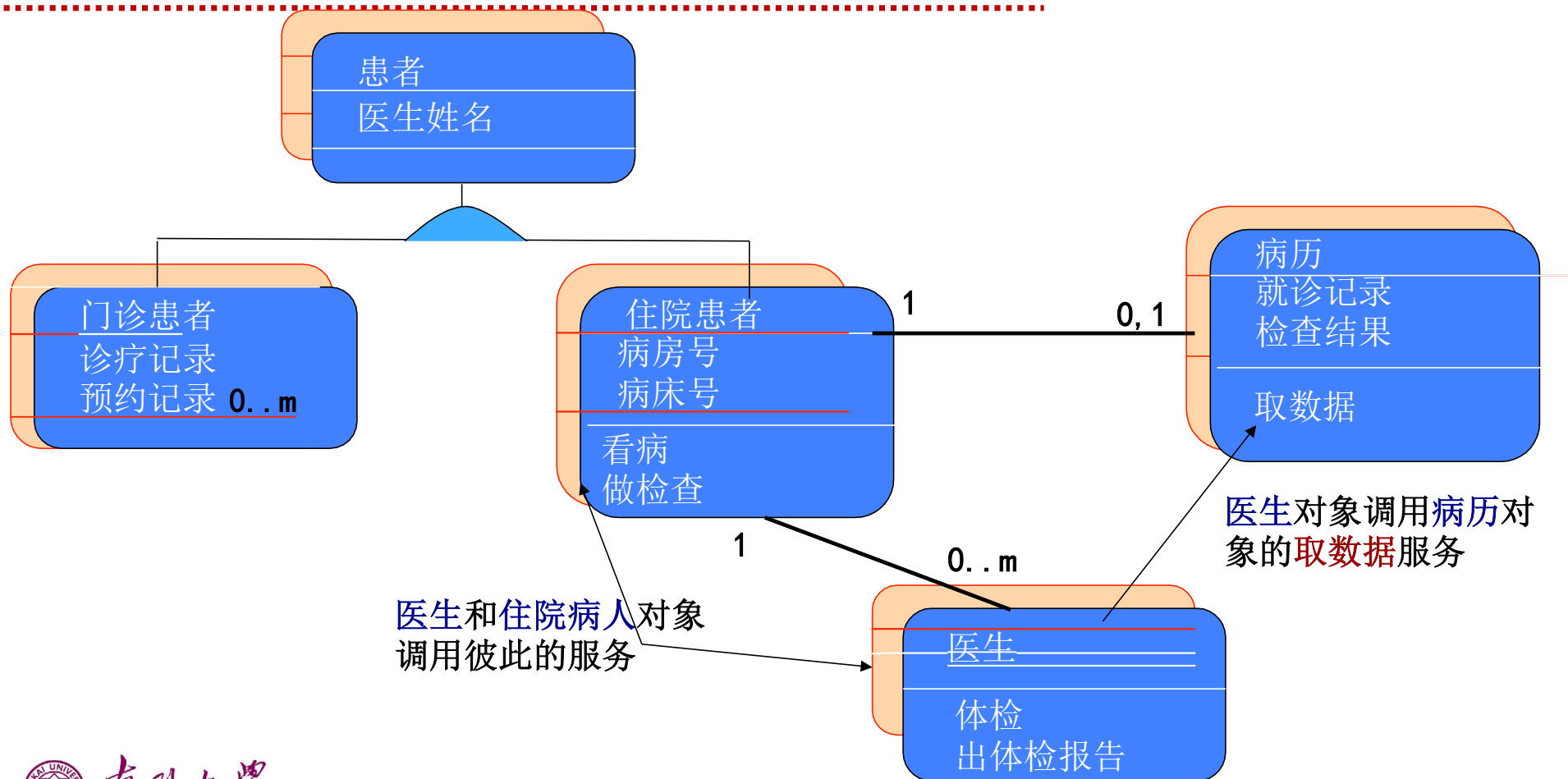
- 注意:
 - **投影和分解有共同点:**
 - 分解定义整体-部分关系
 - 投影定义视图
 - **分解的假设是子模块间依赖性较小**

面向对象的分析方法学

- 识别对象和类（类是对象的抽象定义）
- 识别类之间的关系，建立由继承和组合关系组成的类层次结构
- 定义主题，通过主题将对象模型组织成多个抽象层次或视角，一般说来通过继承关系或整体部分关系联系起来的类同属于一个主题
- 识别各个对象内部的属性信息，并将其赋予相应抽象层次的类
- 为每个类定义服务

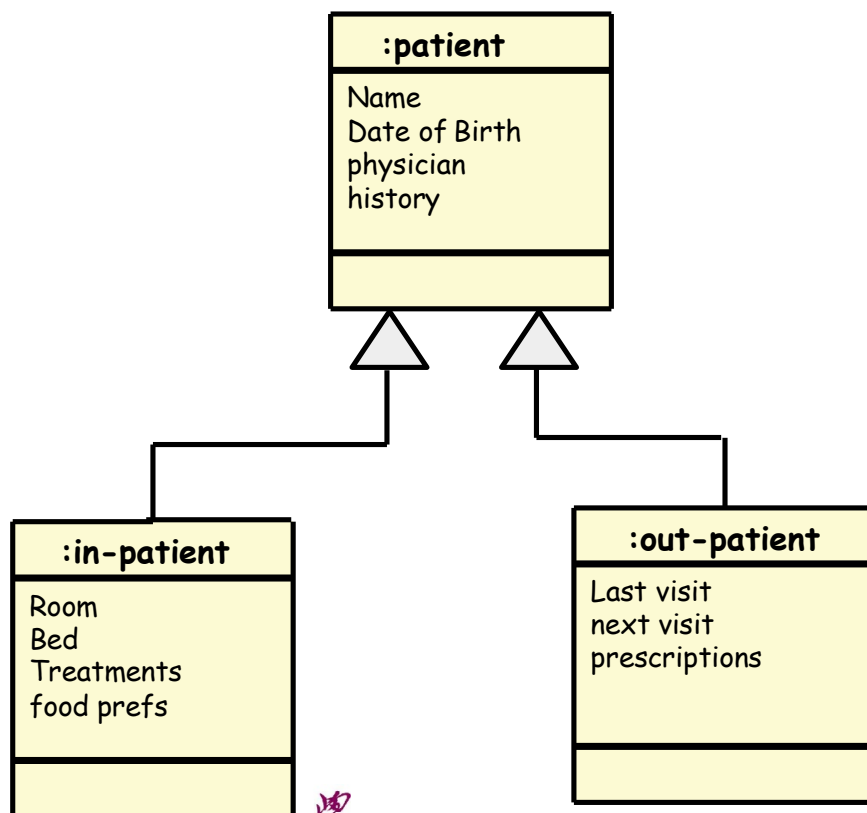


面向对象分析的示例

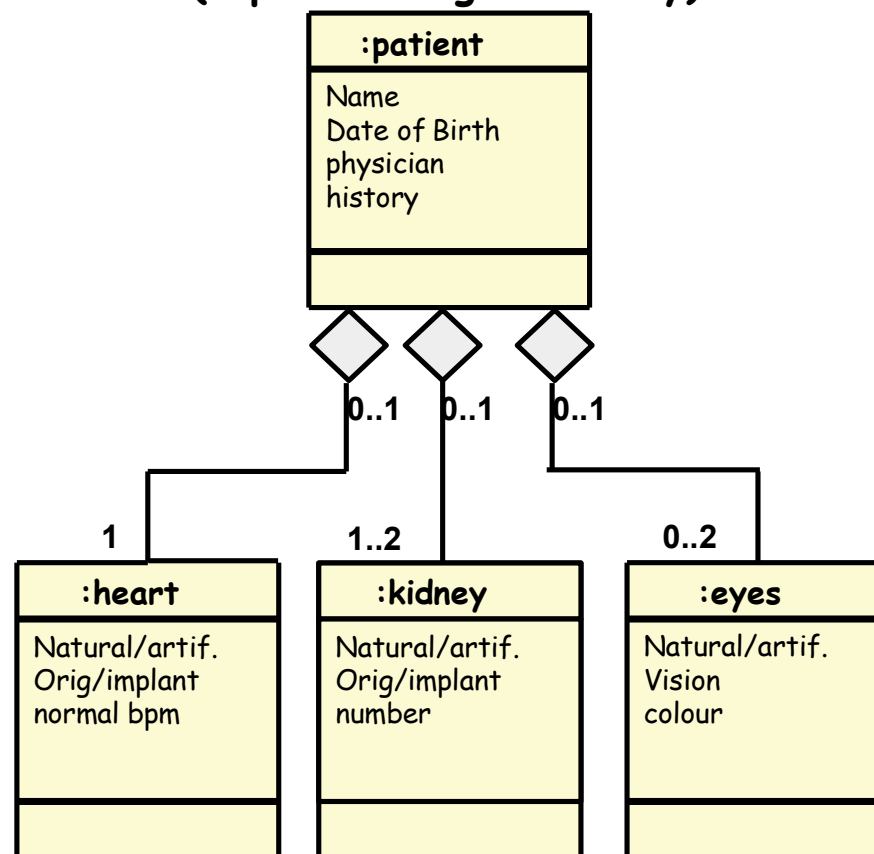


UML类图模型例

继承 Generalization
(an abstraction hierarchy)

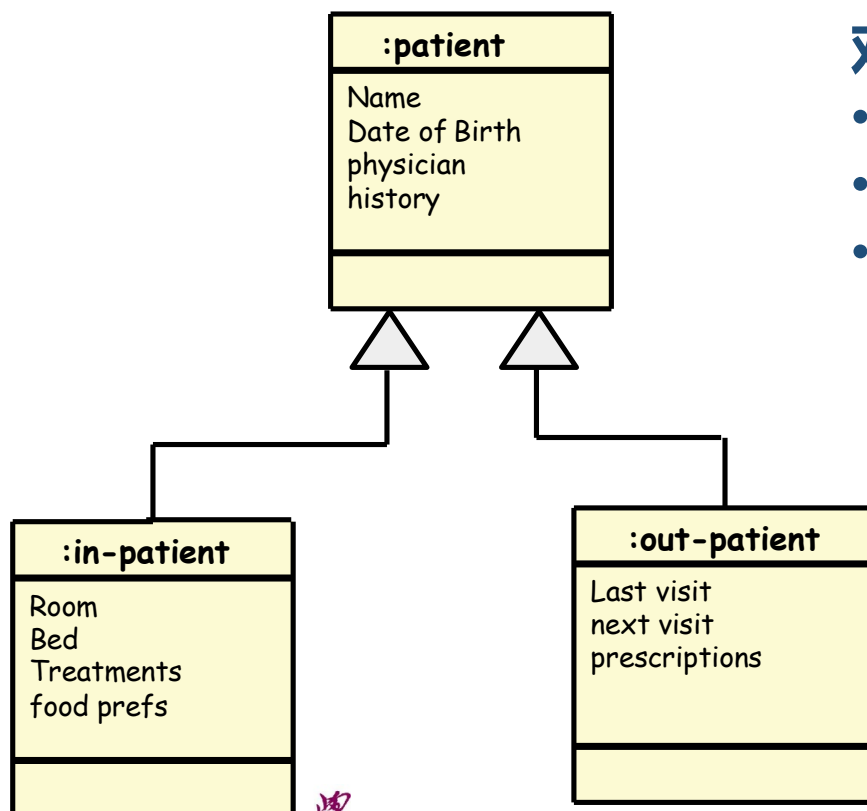


组合 Aggregation
(a partitioning hierarchy)



一般-特殊结构

继承 Generalization
(an abstraction hierarchy)

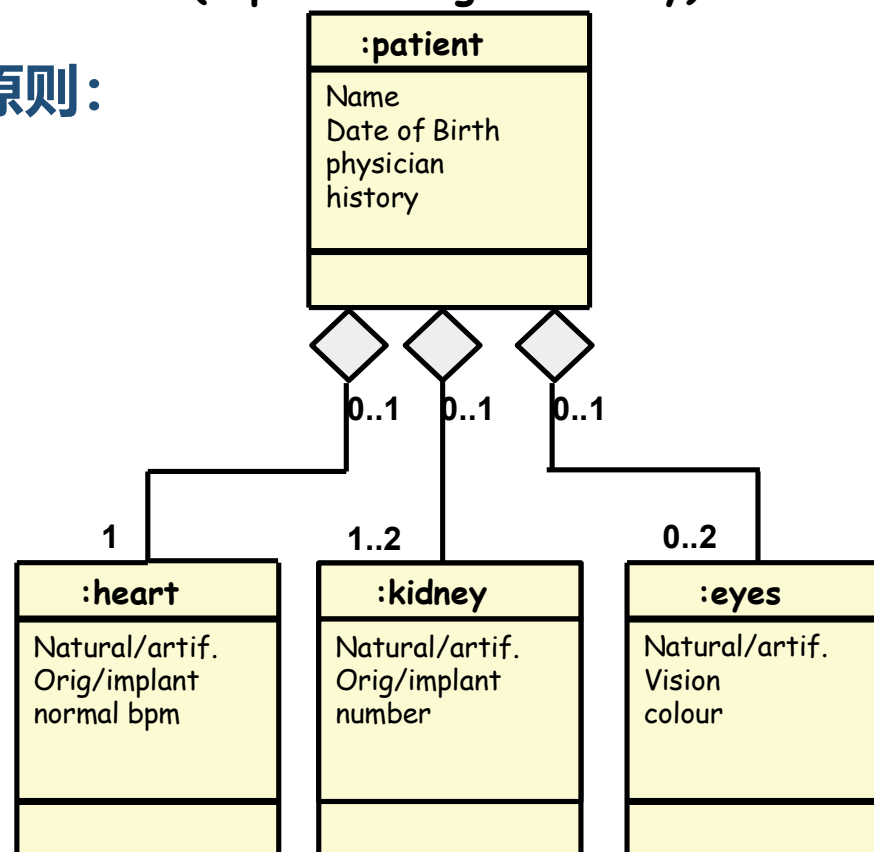


对象建模原则:

- 抽象
- 分解
- 投影

整体-部分结构

组合 Aggregation
(a partitioning hierarchy)





如何识别类 --CRC卡片分拣法

李起成

College of Computer Science
Nankai University

*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University) and
Ivan Marsic (Rugers University)*



南開大學
Nankai University

用例描述

用例名称: 网站公告发布
用例标识号: 202
参与者: 负责人
简要说明: 负责人用来填写和修改家教网站首页的公告, 公告最终显示在家教网站的首页上。
前置条件: 负责人已经登陆家教网站管理系统
基本事件流: 1. 负责人鼠标点击“修改公告”按钮 2. 系统出现一个文本框, 显示着原来的公告内容 3. 负责人可以在文本框上修改公告, 也可以完全删除, 重新写新的公告 4. 负责人编辑完文本框, 按“提交”按钮, 首页公告就被修改 5. 用例终止
其他事件流 A1: 在按“提交”按钮之前, 负责人随时可以按“返回”按钮, 文本框的任何修改内容都不会影响网站首页的公告
异常事件流: 1. 提示错误信息, 负责人确认 2. 返回到管理系统主页面
后置条件: 网站首页的公告信息被修改
注释: 无



识别类的方法

- 根据用例描述中的名词确定类的候选者
- 使用CRC分析法寻找类。
 - CRC是类 (Class)、责任 (Responsibility) 和协作 (Collaboration) 的简称，CRC分析法根据类所要扮演的职责来确定类

CRC卡片

Class: 类定义		Classname 类名	
	Responsibilities 类职责定义	Collaborations 交互协作关系定义	



一个游戏

- 我们要编写一个21点游戏的程序
- 客户交给你一份详细的问题描述，也就是21点游戏规则说明
- 下面我们开始设计这个系统
- 包括识别类、类的职责、类之间的交互行为



21点游戏 (Blackjack)

- 21点游戏的目标是获得点数最接近或等于21的手牌
- 胜者需要赢过庄家的手牌，而且不能超过21点
- 无论桌上有几名玩家，总是只和庄家论输赢
- 21点游戏中，2-10的牌其点数就是牌面的数值
- A的点数根据玩家需要可以取1或11
- J, Q, K的点数都是10



21点游戏 (Blackjack)

- 游戏开始时，玩家首先下注，通常下注的额度是有上限的
- 如果你是唯一玩家，下注后，庄家给你发两张牌，牌面向上，双方均能看到
- 庄家给自己发两张牌，一张牌面向上（明牌），一张牌面向下（底牌）
- 如果有多个玩家，则庄家给每位玩家都是先发一张明牌，庄家自己发一张底牌，再给每个玩家和庄家各发一张明牌



21点游戏 (Blackjack)

- 发完两张牌之后，每位玩家自己决定是否要更多的牌，当对手上的牌数满意之后，就停手等待
- 停手之后，庄家开始加牌，如果庄家持牌总点数少于17，则必须要牌，直到超过16；如果庄家的总点数等于或多于17点，则必须停牌
- **玩家赢：**
 - 玩家的手牌点数超过庄家，且不超过21点
 - 庄家冒过21点
- **和局：**玩家和庄家点数持平，则不输不赢，庄家将玩家赌注归还
- **玩家输：**
 - 玩家冒过21点
 - 庄家的点数超过玩家，且不超过21点

识别21点游戏中的对象类

- 类对象通常对应一个命名实体，因此，用名词表达，于是我们先从名词开始

- 切记：别想一步到位



21点游戏 (Blackjack)

- 21点游戏的目标是获得点数最接近或等于21的手牌
- 胜者需要赢过庄家的手牌，而且不能超过21点
- 无论桌上有几名玩家，你总是只和庄家论输赢
- 21点游戏中，2-10的牌其点数就是牌面的数值
- A的点数根据玩家需要可以取1或11
- J, Q, K的点数都是10



21点游戏 (Blackjack)

- 游戏开始时，玩家首先下**注**，通常下注的额度是有**上限**的
- 如果你是唯一玩家，下注后，庄家给你发两张牌，牌面向上，双方均能看到
- 庄家给自己发两张牌，一张牌面向上（**明牌**），一张牌面向下（**底牌**）
- 如果有多个玩家，则庄家给每位玩家都是先发一张明牌，庄家自己发一张底牌，再给每个玩家和庄家各发一张明牌



21点游戏 (Blackjack)

- 发完两张牌之后，每位玩家自己决定是否要更多的牌，当对手上的牌数满意 之后，就停手等待
- 停手之后，庄家开始加牌，不能超过21点
- 当你的手牌点数超过庄家时，你赢，当你的手牌点数不及庄家时，庄家赢
- 玩家赢：
 - 你的手牌点数超过庄家，切不超过21点
 - 庄家冒过21点
- 玩家和庄家点数持平，则不输不赢，庄家将玩家赌注归还
- 玩家输：
 - 玩家冒过 21点

通过名词过滤识别出的对象类

- 游戏
- 21点
- 庄家
- 玩家
- 牌
- 牌堆
- 手牌
- 点数
- 花色
- 赢家
- A
- 数字牌
- K
- Q
- J
- 赌注



类筛选

- 在候选类中排除以下类:
 - 超出问题关注的范围的类;
 - 指代整个系统的类;
 - 功能重复的类;
 - 过于含糊或过于具体的类
 - 可观察到的现象是, 实例对象过多过少
- Coad & Yourdon' s 的筛选原则:
 - **保存对象信息**: 系统需要保存对象信息吗?
 - **提供所需服务**: 类对象是否对外提供修改属性值的操作?
 - **具有多个属性**: 只有一个属性的类, 应该建模为属性
 - **具有公共属性**: 类属性是否为所有实例对象共享?
 - **具有公共操作**: 类操作是否为所有实例对象共享?
 - **外部实体**: 如果生产或使用对象的信息, 也应考虑建模为系统类



由此识别出的对象

- 游戏
- 21点
- 庄家
- 玩家
- 牌
- 牌堆
- 手牌
- 点数
- 花色
- 赢家
- A
- 数字牌
- K
- Q
- J
- 赌注



选出的类

- 牌、牌堆、手牌、庄家、玩家、赌注

Card

Deck

Hand

Dealer

Player

Bet



类识别

- **从原始资料中识别类:**

- 找出干系人提交的问题描述中名词及短语
 - 如果他描述应用领域中的信息结构或本质，则加入模型

- **从其他来源识别类:**

- 背景信息调查
- 用户及干系人提供
- 分析模式

- **最好识别出尽可能多的候选类**

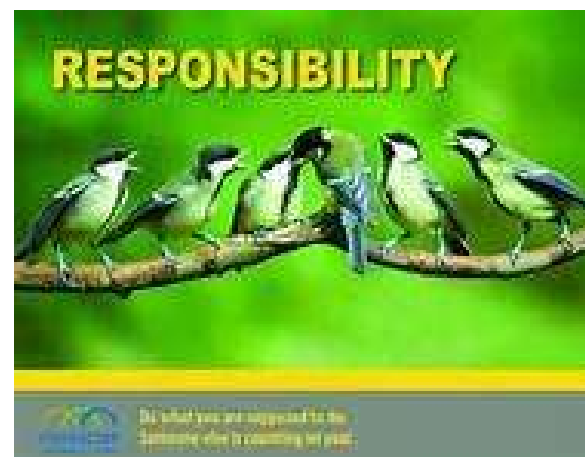
- 之后逐步按照其价值功用进行选择
- 明确判断后排除一个类要比压根不考虑来的合理

识别类的功能职责

功能职责关乎行为动作，因此是问题描述中的动词

注意：

1. 并非所有动词均将成为类职责
2. 有时多个动作合并为一个职责
3. 随着分析过程深入会发现新的职责
4. 不断修正类定义和职责定义
5. 当两个类分享职责时，为二者同时添加该职责



21点游戏 (Blackjack)

- 游戏开始时，玩家首先下注，通常下注的额度是有上限的
- 如果你是唯一玩家，下注后，庄家给你发两张牌，牌面向上，双方均能看到
- 庄家给自己发两张牌，一张牌面向上（明牌），一张牌面向下（底牌）
- 如果有多个玩家，则庄家给每位玩家都是先发一张明牌，庄家自己发一张底牌，再给每个玩家和庄家各发一张明牌



21点游戏 (Blackjack)

- 发完两张牌之后，每位玩家自己**决定**是否**要**更多的**牌**，当对手上的牌**点数满 意**之后，就停手等待
- 停手之后，庄家开始**加牌**，不能超过21点
- 当你的手牌点数超过庄家时，你赢，当你的手牌点数不及庄家时，庄家赢
- 玩家赢：
 - 你的手牌**点数超过**庄家，切不超过21点
 - 庄家冒过21点
- 玩家和庄家点数持平，则不输不赢，庄家将玩家赌注**归还**
- 玩家输：
 - 玩家冒过 21点

候选类对应的功能职责

- 牌 (Card)
 - 读取牌面数字
 - 读取花色
 - 读取点数
 - 判断是否数字牌
 - 判断是否为A
 - 判断是否为 J、Q、K
- 牌叠 (Deck)
 - 洗牌
 - 发牌
 - 查询剩余牌数
 - 是否能够新开一叠牌
- 赌注 (Bet)
 - 赌注类型
 - 赌注的大小
 - 剩余赌资
 - 判断赌资是否够下注
- 手牌 (Hand)
 - 读取手中牌数
 - 读取手牌总点数
 - 亮牌

候选类对应的功能职责

• 庄家

- 发牌
- 洗牌
- 发一张牌给玩家
- 庄家亮牌
- 计算庄家手牌点数
- 查询庄家手牌点数
- 加一张牌
- 确定赢家
- 开始下一轮游戏

• 玩家

- 请求发牌
- 玩家亮牌
- 计算玩家手牌点数
- 查询玩家手牌点数
- 查看手牌点数是否超过21点
- 查看手牌点数是否等于21点
- 查看手牌点数是否低于21点



识别类交互协作关系：用UML用例图

- 识别对象及其消息交互
- 注意：目的并非写出所有场景，而是对类和职责定义进行精化...



情景举例

庄家

- 庄家洗牌 ✓
- 庄家发第一张牌 ✓
- 庄家给自己发牌 ✓
- 手牌返回庄家手牌点数 ✓
- 庄家发第二张牌 ✓
- 庄家询问玩家是否继续要牌 ✓
- 庄家询问玩家手牌点数 ✓
- 庄家亮牌 ✓
- 庄家分配赌金 ✓



玩家

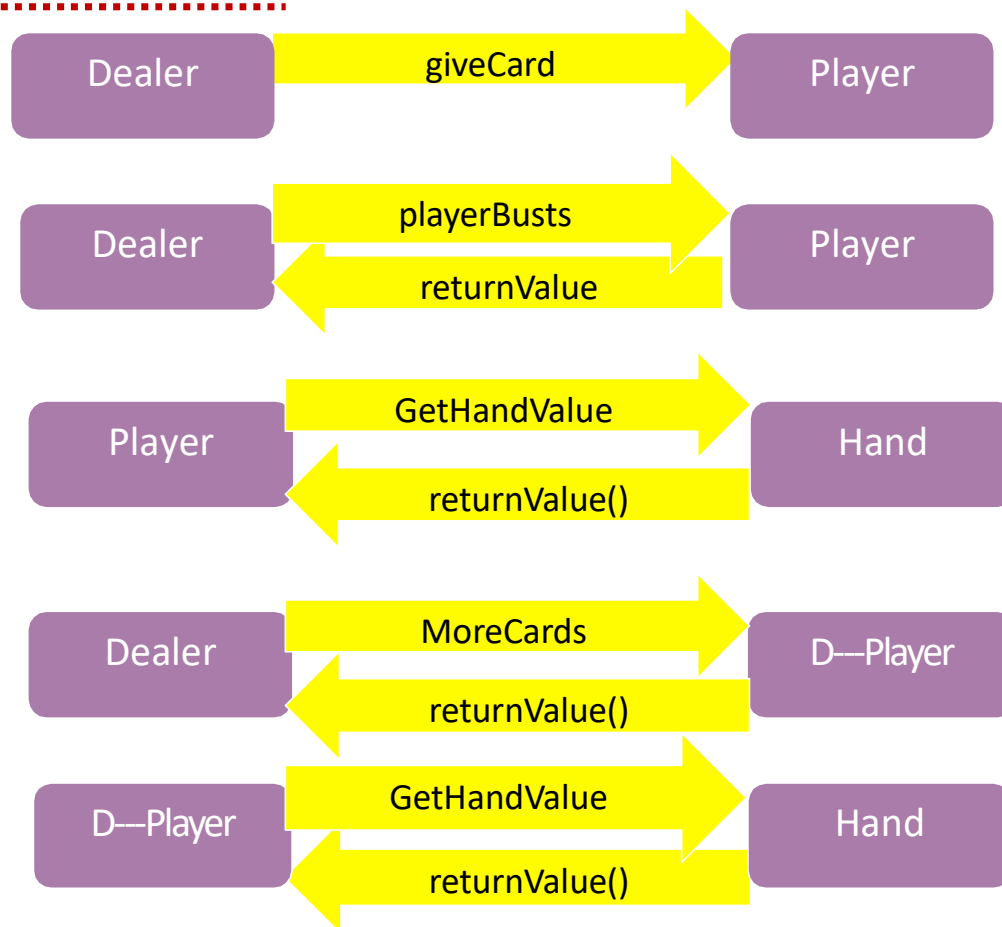
- 玩家下注 ✓
- 玩家拿牌到手里 ✓
- 手牌返回玩家手牌点数 ✓
- 玩家继续要牌 ✓
- 玩家拿牌到手里 ✓
- 手牌返回玩家手牌点数 ✓
- 玩家增加或减少赌注 ✓
- 玩家亮牌 ✓



识别交互关系



最终获得的CRC卡片!



类定义：纸牌类 (Card)

类名 (Class)		纸牌 Card	
	功能职责 (Responsibilities)		交互类 (Collaborations)
	GetName		Deck
	GetValue		



类定义：牌堆类 (Deck)

类名 (Class)

Deck

功能职责 (Responsibilities)

Reset deck

Get deck size

Get next card

Shuffle Deck

Show deck

交互类 (Collaborations)

Dealer

Card



类定义：庄家 (Dealer)

类名 (Class)

Dealer

功能职责 (Responsibilities)

Start a new game

Get a card

交互类 (Collaborations)

Hand

Player

Deck



类定义：玩家 (Player)

类名 (Class)

Player

功能职责 (Responsibilities)

Want more cards

Get a card

Show hand

Get value of hand

交互类 (Collaborations)

Dealer

Hand



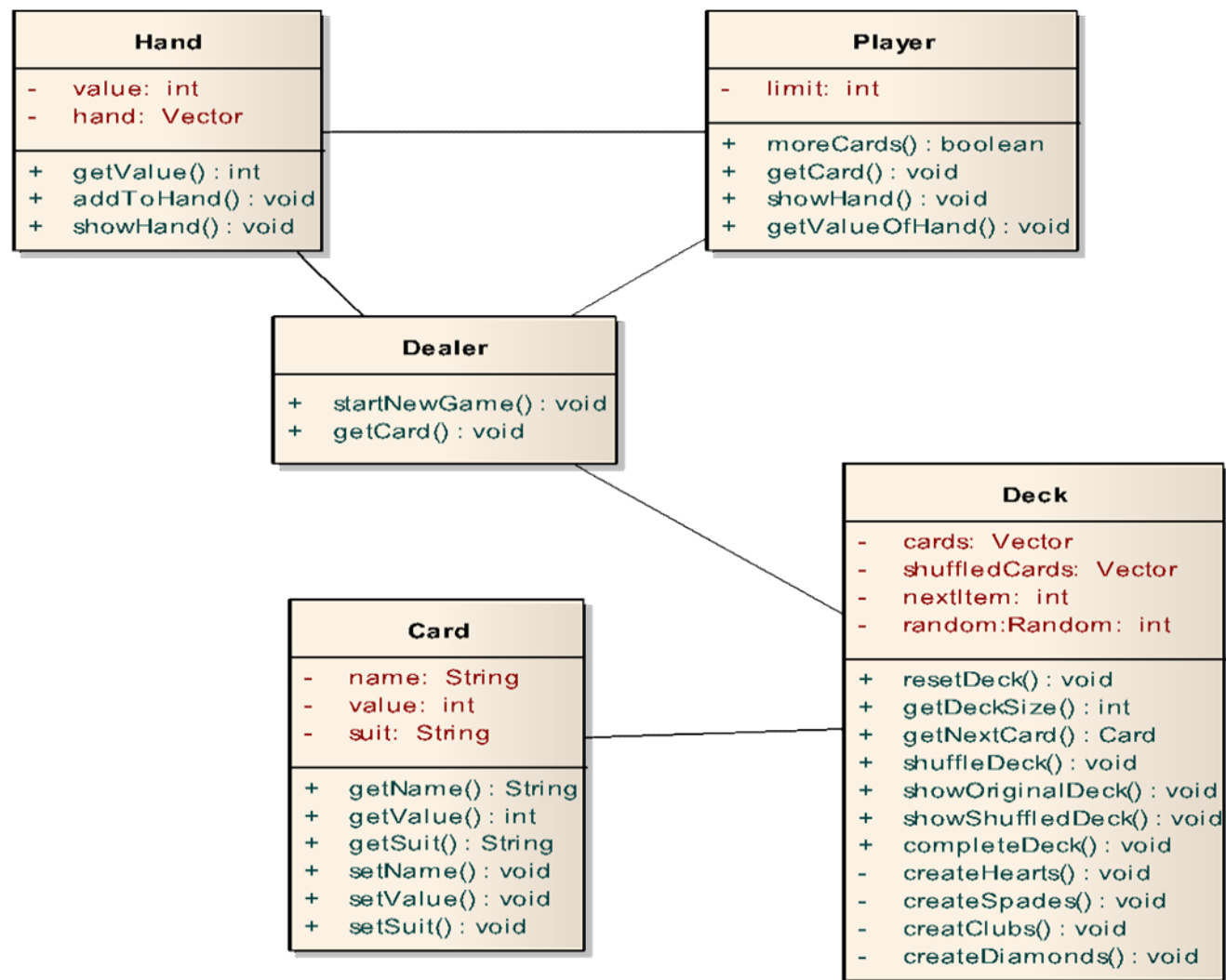
类定义：手牌类 (Hand)

类名 (Class)		Hand	
	功能职责 (Responsibilities)		交互类 (Collaborations)
	Return Value		Dealer
	Add a card		Player
	Show hand		



UML 类图

class system



实体类

- 实体类是用于对必须存储的信息和相关行为建模的类。实体对象（实体类的实例）用于保存和更新一些现象的有关信息。
- 例如：事件、人员或者一些现实生活中的对象
- 实体类通常都是永久性的，它们所具有的属性和关系是长期需要的，有时甚至在系统的整个生存期都需要。

边界类

- 边界类是系统内部与系统外部的业务主角之间进行交互建模的类。
- 边界类是系统为业务主角操作提供的一个GUI，或者系统与其他系统之间进行一个交互的接口。所以当外部的GUI变化时，或者是通信协议有变化时，只需要修改边界类就可以了，不用再去修改控制类和实体类
- **用户界面类** - 帮助与系统用户进行通信的类
- **系统接口类** - 帮助与其他系统进行通信的类
- **设备接口类** - 为用来监测外部事件的设备（如传感器）提供接口的类

控制类

- 控制类用于对一个或几个用例所特有的控制行为进行建模，它描述的用例的业务逻辑的实现，控制类的设计与用例实现有着很大的关系。
- 当业务主角通过边界类来执行用例的时候，产生一个控制类对象，在用例被执行完后，控制类对象会被销毁。
- 控制类的特点：
 - 独立于环境
 - 和用例的实现关联
 - 使用关联实体类，操作实体类对象
 - 专注于业务逻辑的实现



面向对象设计

Object-Oriented Design

李起成

College of Computer Science
Nankai University

*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University) and
Ivan Marsic (Rugers University)*



南开大学
Nankai University

关于对象设计

- 其实，真相是：

- 面向对象方法本身并不能保证你的设计成为优秀的设计

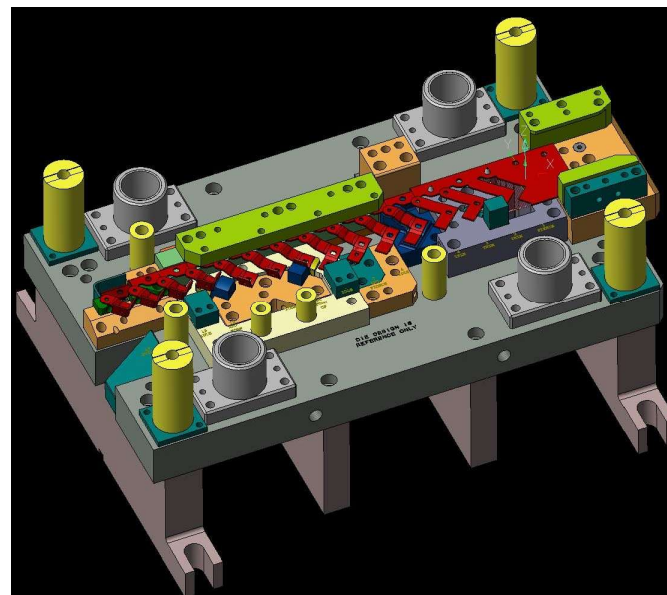
- You can create a very bad OO design

- just as easily as you can create a very bad non—OO design...



面向对象设计过程

- 进行适当的领域分析
- 撰写问题描述，确定系统的开发任务
- 基于问题描述抽取需求
- 开发用户界面原型
- 识别对象类
- 定义每个类的职责
- 确定类之间的交互关系
- 建立系统的设计模型



面向对象思维式的核心理念

- 区分接口与实现
- 从具体到抽象
- 接口设计原则



区分接口与实现

• 接口的标准化 vs. 实现的演化

class System

DataBaseReader

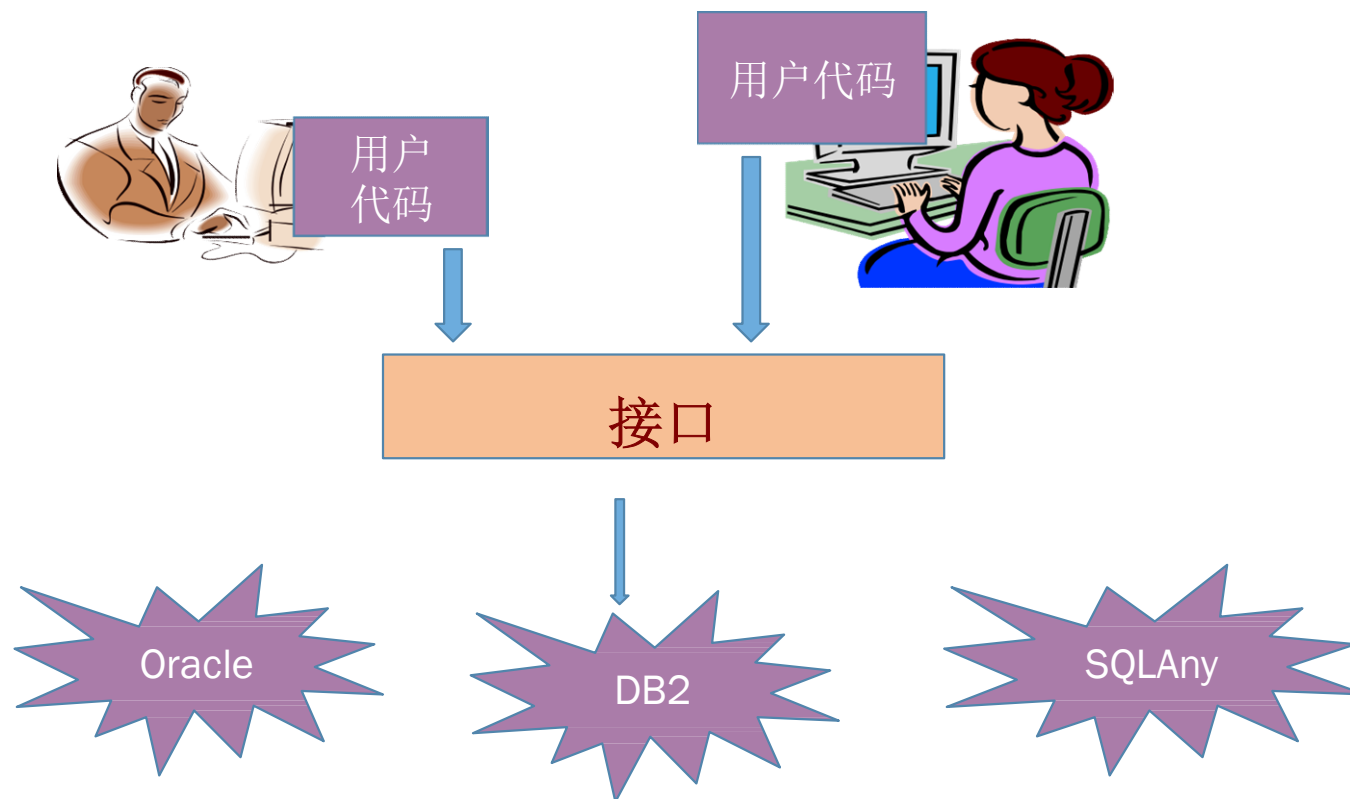
```
+ open() : void
+ close() : void
+ goToFirst() : void
+ goToLast() : void
+ howManyRecords() : int
+ areThereMoreRecords() : boolean
+ positionRecord() : void
+ getRecord() : char
+ getNextRecord() : char
```

```
public void open(string name) {
    /* some application-specific processing */
    /* call the Oracle API to open the DB*/
    /* more application specific processing */
}
```

```
public void open(string name) {
    /* some application-specific processing */
    /* call the SQLAnywhere to open the DB*/
    /* more application specific processing */
}
```



接口



设计抽象的接口

Using Abstract Thinking When Designing Interfaces



抽象的接口



抽象的接口



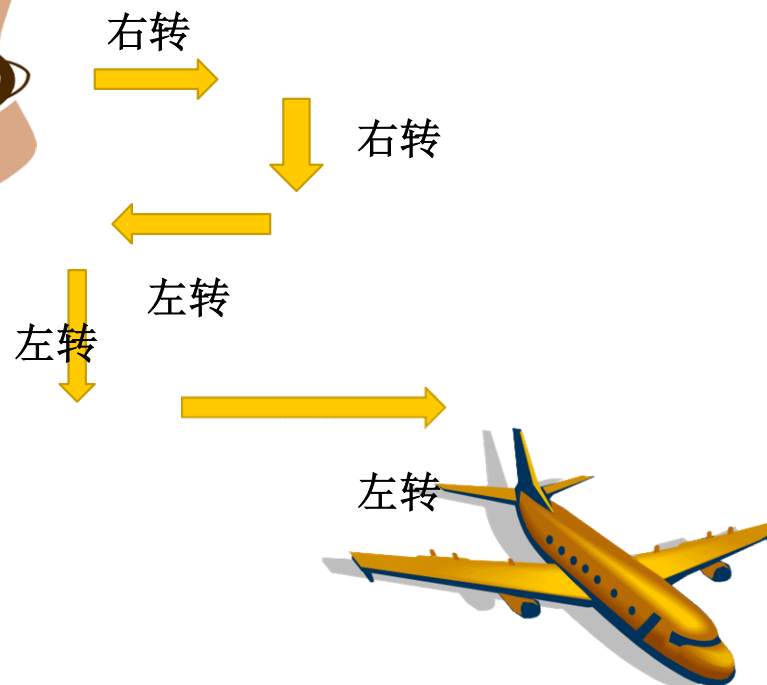
师傅，请送我去机场



不太抽象的接口



不够抽象的接口



确定用户

- 用户是谁？ 重要程度高达50
- **面向服务的原则 (Services Principle)**



提供服务:
只要能赚钱就好



使用服务:
不要太贵喔



公共接口的识别

- 用户使用出租车对象的时候，需要以下功能：
 - 上车
 - 告知司机终点
 - 付钱
 - 下车
- 用户要用出租车的时候：
 - 有出行地点
 - 召唤出租车
 - 付钱



class System

Cabbie

```
+ hailTaxi() : void
+ enterTaxi() : void
+ greetCabbie() : void
+ specifyDestination() : void
+ payCabbie() : void
+ tipCabbie() : void
+ leaveTaxi() : void
```



确定对象行为

用例!!!

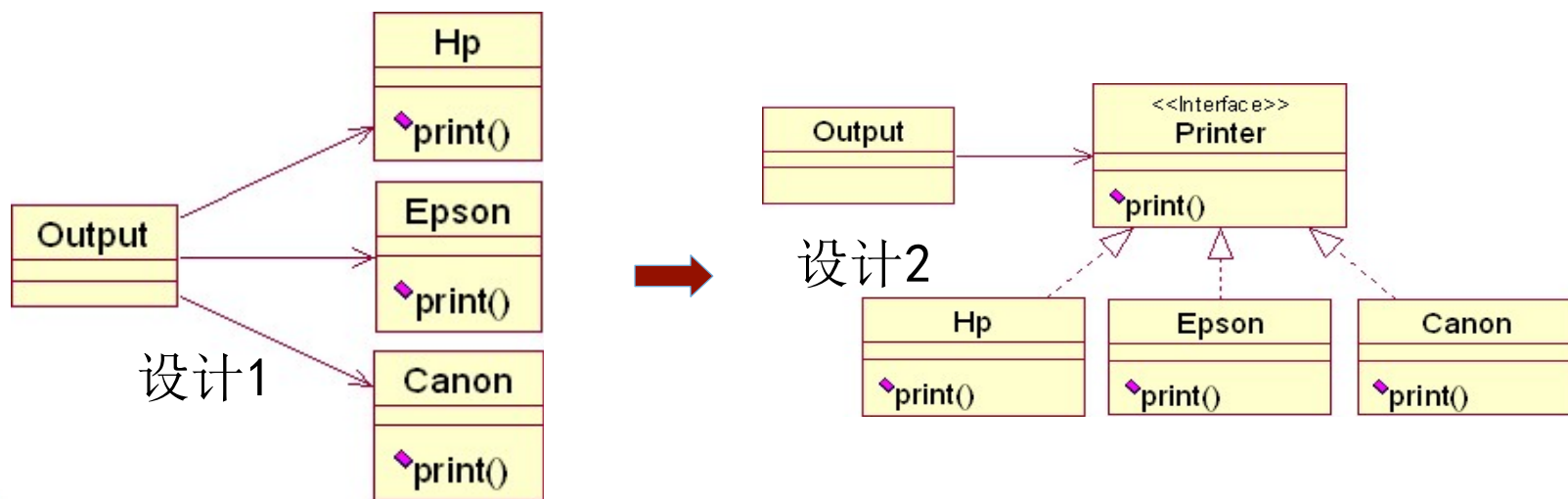
以往的设计决策在我们定义抽象接口时候会发生变化

确定实现细节

- 公共接口以外的内容都可以看做是实现相关的
- 用户永远无需关注实现细节
- 对实现的修改无须牵涉接口
- 实现为用户的期望提供解决方案
- 接口从用户的角度看待对象，实现则是对象的果核和果肉
- 实现中包含有描述对象状态的代码

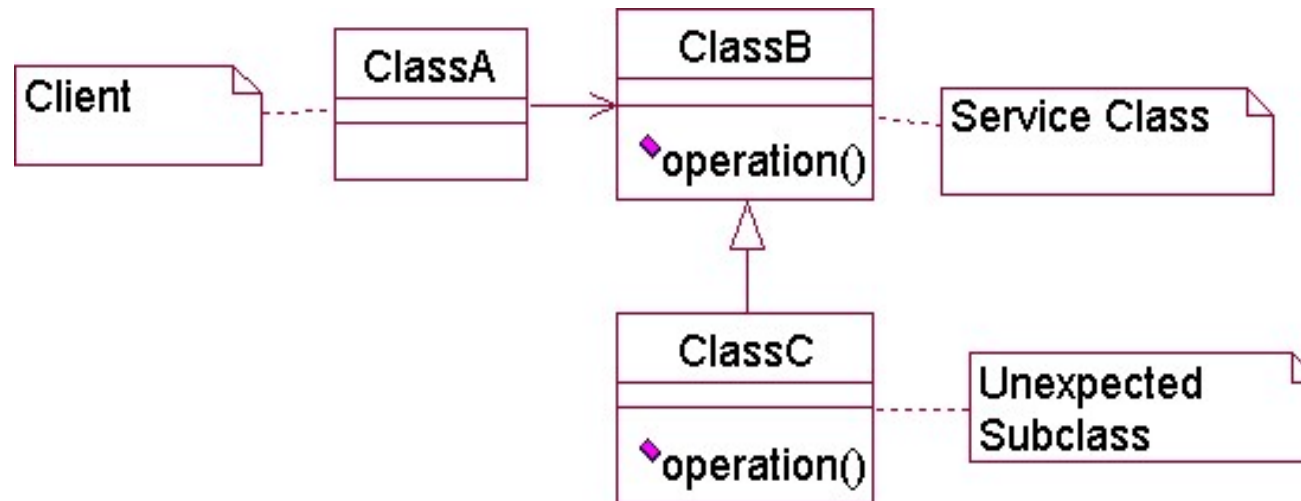
开闭原则(Open/Closed Principle, OCP)

- 最初由Bertrand Meyer提出
- 软件实体在扩展性方面应该是开放的，而在更改性方面应该是封闭的。
- 例：打印输出设计



Liskov Substitution Principle (LSP)

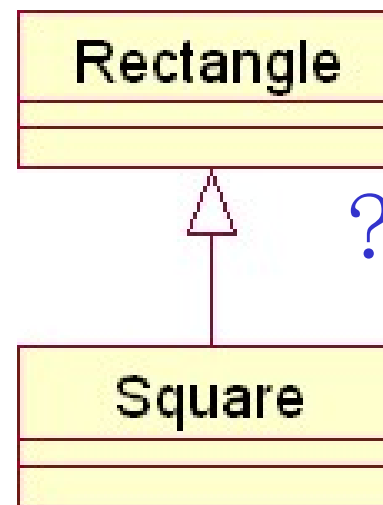
- 最早由Liskov于1987年在OOPSLA会议上提出
- 子类可以替换父类出现在父类能出现的任何地方



违反LSP的例子

```
public class Rectangle
{
    private int topLe5X;
    private int topLe5Y;
    int width;
    int height;

    public void setWidth(int width)
    {
        this.width = width;
    }
    public void setHeight(int height)
    {
        this.height = height;
    }
    public int getWidth()
    {
        return width;
    }
    public int getHeight()
    {
        return height;
    }
}
```

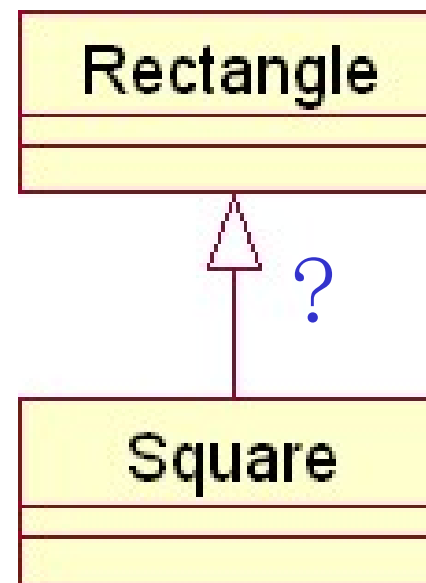


问题：如果将Square作为Rectangle的子类，则如何定义Square类的setWidth和setHeight方法？



解决方法

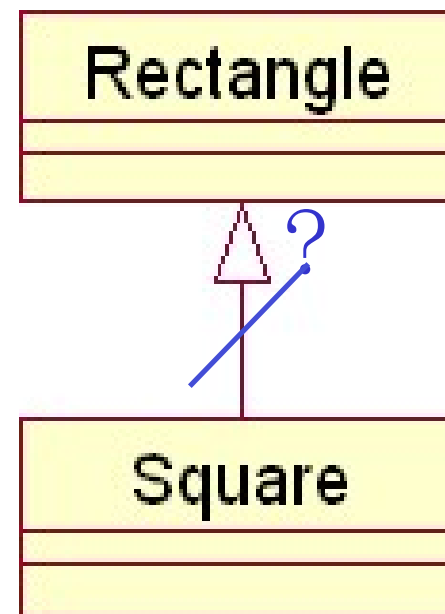
```
public class Square extends Rectangle
{
    public void setWidth(int width) {
        super.setWidth(width);
        super.setHeight(width);
    }
    public void setHeight(int height)
    {
        super.setHeight(height);
        super.setWidth(height);
    }
}
```



这种解决方法是否可行？

考虑下面的代码：

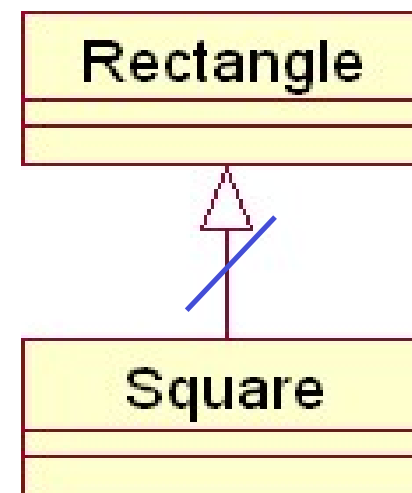
```
public class Test {  
    public static void main(String[] args)  
    {  
        Test t = new Test();  
        Rectangle r = new Rectangle();  
        Square s = new Square();  
        t.g(r);  
        // t.g(s);  
    }  
    private void g(Rectangle r) {  
        r.setWidth(10);  
        r.setHeight(20);  
        assert (r.getWidth()*r.getHeight() == 200);  
    }  
}
```



如果传给方法g的参数是Rectangle类型的对象，则没有问题，如果是Square类型的对象，则出错。

思考题：如何知道子类的行为符合父类的要求？

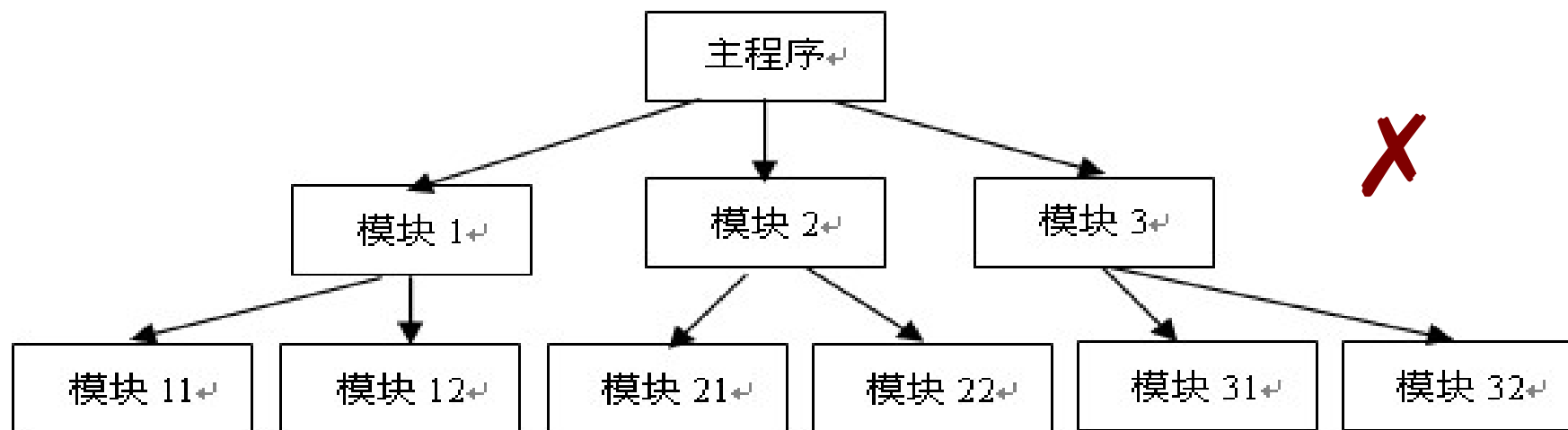
- 契约式设计 (Design by Contract)
- 为了满足Liskov替换原则，设计时要求：
- 子类中方法的前置条件不能强于父类中相应方法的前置条件。
- 子类中方法的后置条件不能弱于父类中相应方法的后置条件。



Liskov 替换原则要求子类宽入严出！

依赖倒置原则 (Dependency Inversion Principle, DIP)

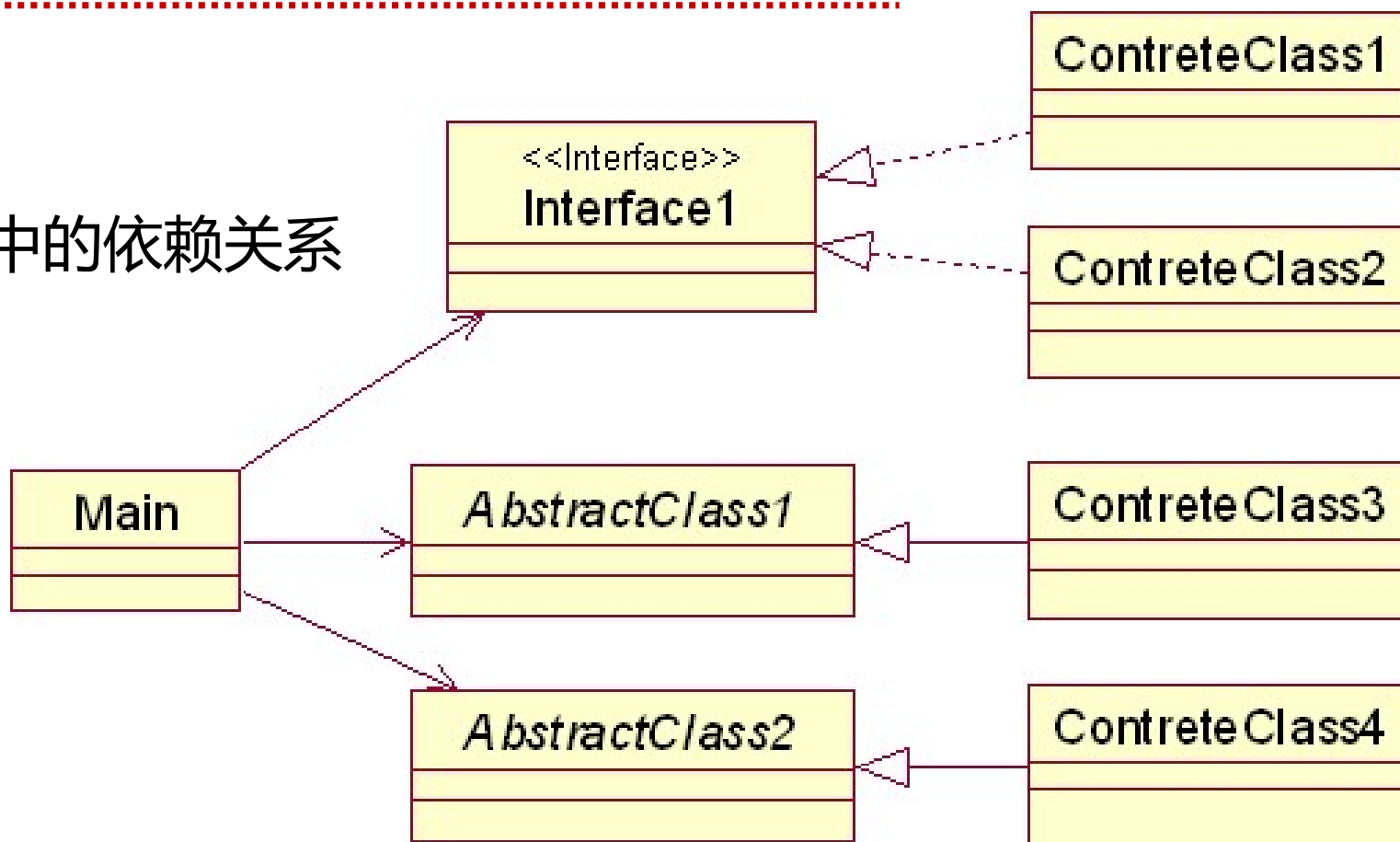
- 依赖倒置原则指的是依赖关系应该是尽量依赖接口(或抽象类), 而不是依赖于具体类。



结构化设计中模块间的依赖关系

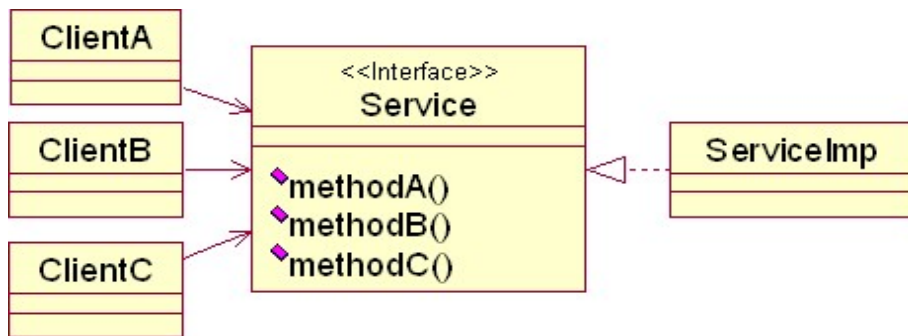
面向对象设计中的依赖关系

OOD中的依赖关系

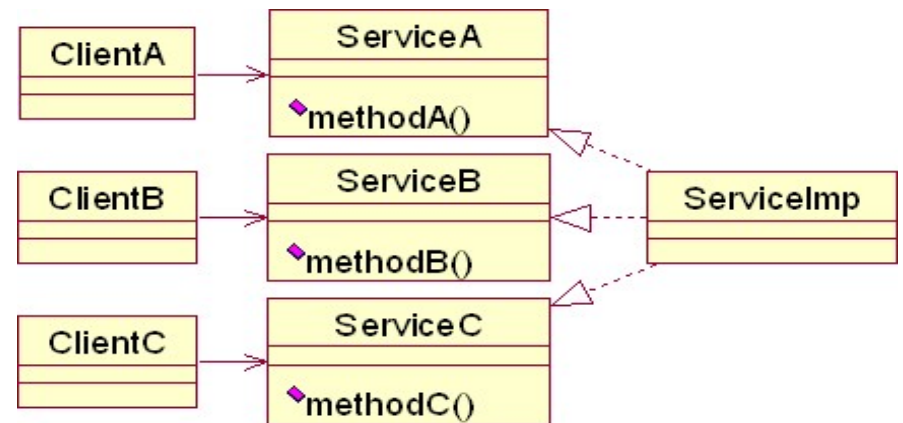


接口分离原则 (Interface Segregation Principle, ISP)

- 在设计时采用多个和特定客户类 (client) 有关的接口要比采用一个通用的接口要好。



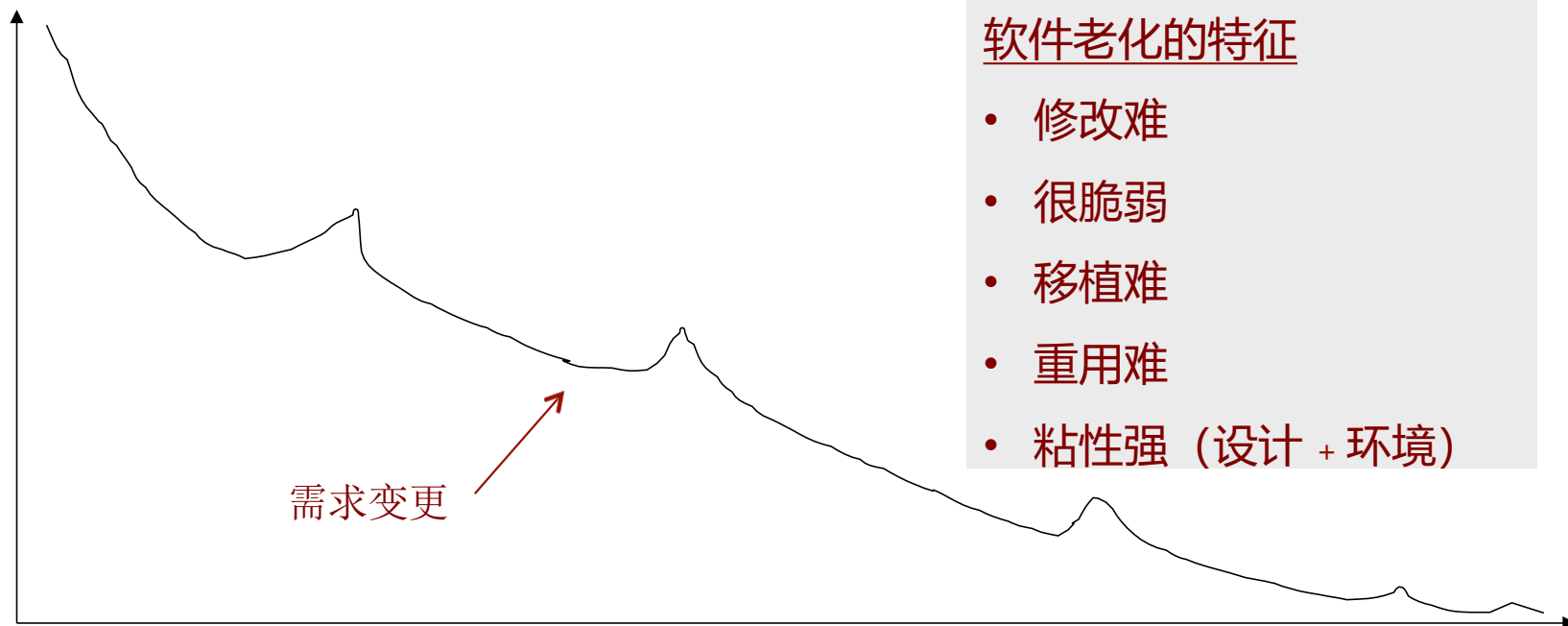
使用通用接口的设计



使用分离接口的设计

好的系统设计的特征

- 用户友好
- 易理解
- 可靠
- 可扩展
- 可移植
- 可伸缩
- 可重用



软件老化的特征

- 修改难
- 很脆弱
- 移植难
- 重用难
- 粘性强 (设计 + 环境)

• ...

简单性：实现简单，使用简单，理解简单，维护简单

○○设计时要注意的一些问题

1. 不同类中相似方法的名字应该相同
2. 遵守已有的约定俗成的习惯
3. 尽量减少消息模式的数目。只要可能，就使消息具有一致的模式，以利于理解。
4. 设计简单的类。类的职责要明确，应该从类名就可以较容易地推断出类的用途。
5. 定义简单的操作、方法
6. 定义简单的交互协议
7. 泛化结构的深度要适当
8. 把设计变动的副作用减至最少